Testing Digital Signal Processing Code without an Oracle: Metamorphic Relations, Domain Generators and Error-Model Tolerances

Clément Basdevant
Independent Researcher
ORCID 0009-0004-7496-2365

Nicolas Boganski
[Affiliation to be completed]

***Abstract*—Signal processing code is hard to test because the correct output of a filter, a transform or a resampler on an arbitrary input is unknown. Libraries therefore compare a few hand-picked cases with values computed by another tool, under tolerances chosen by hand: in the test suite of `scipy.signal` alone we count 66 distinct hand-written tolerances spanning 31 orders of magnitude, and no test in the five suites we surveyed generates its inputs. We present dsp-props, an open-source Python library that tests signal processing operators against properties every correct implementation must satisfy (linearity, shift and convolution theorems, Parseval's identity, causality, consistency with a declared transfer function, periodic time invariance of resamplers, invariance of streaming processors to block boundaries), on inputs drawn from signal families known to break such code, with tolerances derived from an error model of each operation rather than chosen by hand. On 183 automatically generated faults in four reference implementations, dsp-props detects 90 % of the non-equivalent faults and test suites ported from NumPy and SciPy detect 96 %; the two miss different faults and together detect all of them. On 31 hand-written faults, every variant detects every fault, and ablations show that the domain generators and the error-model tolerances make failures appear sooner, with smaller counterexamples, and without false alarms on subnormal inputs. On correct implementations it raised no false alarm in 2 000 runs. Applied to SciPy, NumPy, PyTorch, librosa and torchaudio, it found faults that were reported upstream.**

***Index Terms*—software testing, metamorphic testing, property-based testing, digital signal processing, floating-point error, test oracle**

I. Introduction

Digital signal processing (DSP) code sits at the bottom of a large software stack: the fast Fourier transform (FFT), linear filters and sample-rate converters of NumPy [1], SciPy [2] and PyTorch [3] are called by audio, communication, geophysics and machine-learning code that rarely re-checks them. Testing these operators faces three problems at once.

P1, no oracle: For most inputs nobody knows the exact output of a filter or a transform. This is the *oracle problem* of software testing [4], and it is acute for scientific software [5]. In practice, tests compare against values computed by another tool and typed into the test file, or against a second implementation that may share the same assumptions. In the suites we surveyed (Section III), 169 tests of `scipy.signal` compare with hard-coded arrays.

P2, inputs that do not exercise the faults: Hand-written tests use a few lengths and smooth or random signals. Faults in DSP code, however, hide in specific places: lengths that are prime, odd or one away from a power of two (which change the FFT algorithm), tones between two DFT bins or close to the Nyquist frequency, extreme amplitudes, and, for streaming code, empty or one-sample blocks. A concrete case: `scipy.signal.lfilter` returned an uninitialised final state when given an empty block and an initial state (SciPy issue gh-22571), so that any block-processing loop silently diverged after an empty block; `sosfilt` still raises on such input. Across the 2 388 tests we surveyed, only 2 combine an initial state with an empty input, and none generates its inputs.

P3, no principled tolerance: Even when the expected output is known, floating-point results are only close to it, and *how close* depends on the operation, the length and the conditioning of the problem. Tests therefore use tolerances chosen by hand: 66 distinct values in `scipy.signal`, from 10^{-31} to 3.01. A tolerance that is too loose hides a single-precision intermediate; one that is too tight produces false alarms, which teach maintainers to loosen tolerances further.

Property-based testing [6], [7] answers P2 in general by generating inputs, and metamorphic testing [8], [9] answers P1 in general by checking relations between outputs instead of the outputs themselves. Neither tells a DSP developer which relations hold for which operator, which signals to generate, nor how to compare floating-point results; and a test tool that claims to find faults in numerical libraries must itself be shown not to raise false alarms.

This article makes the following contributions.

- A catalogue of metamorphic relations for four classes of DSP operators (transforms, linear time-invariant filters, rational resamplers, streaming processors), each with a proof (Appendix A), selected by a one-line declaration of what the operator is (answers P1).
- Signal generators that target the inputs known to break DSP code (P2).
- Tolerances derived from forward error bounds of the FFT and of linear recursions, and a calibration procedure that measures an implementation against an extended-precision reference but refuses to absorb an implausibly large error (P3).

- An evaluation protocol that compares the approach with the test suites of NumPy and SciPy on automatically generated faults, isolates the contribution of the generators and of the tolerances, and validates the tool itself: false alarms on correct code, margin between observed error and tolerance, accuracy of the reference against 50-digit arithmetic, and an audit of the generators.

The library, the experiments and the data are available at https://github.com/clement-basdevant/dsp-props under a BSD licence.

II. Background and related work

A. Testing without an oracle

Metamorphic testing [8], [9], applied among others to bioinformatics [10] and image processing [11], checks *metamorphic relations*: if f is correct, then for related inputs x and x' the outputs $f(x)$ and $f(x')$ must satisfy a known relation, e.g., $f(x + x') = f(x) + f(x')$ for a linear f. Property-based testing (PBT), introduced by QuickCheck [6] and available in Python through Hypothesis [7], generates the inputs and shrinks a failing input to a minimal one [12]. Both are general techniques: the relations, the generators and the comparison rule are left to the user. Interviews of PBT users identify writing properties and generators as their main weakness [13]; Maaz et al. use language models to propose properties across the Python ecosystem [14].

B. Self-testing of numerical programs

Blum, Luby and Rubinfeld [15] showed that a program computing a linear function can be tested, and even corrected, by checking random linear identities. Ergün [16], [17] extended the idea to multivariate linear functions such as the DFT, and Ergün, Kumar and Rubinfeld [18] to computations that are only approximately correct. The FFTW library [19] ships a self-test built on these results, which checks linearity, impulse responses and time and frequency shifts [20]. These works establish that linearity and shift identities characterise the DFT; they do not cover filters, resamplers or streaming code, and they treat the floating-point error with a single threshold.

C. Differential testing and numerical faults

Differential testing compares several implementations of the same function [21]. It needs at least two of them, and they may share an algorithm or an assumption, e.g., the same handling of an empty block. Di Franco et al. analysed 269 real numerical bugs in NumPy, SciPy, LAPACK, GSL and Elemental [22], showing that such bugs reach widely used, mature libraries.

D. Floating-point error of DSP operators

The FFT computed by a Cooley–Tukey algorithm has a relative error that grows as $O(\varepsilon \log N)$ [23], [24], where ε is the unit roundoff, whereas the direct DFT grows as $O(\varepsilon N)$. The error of a linear recursion depends on the conditioning of its poles [24]. These bounds are rarely used in tests, which fall back on fixed relative tolerances.

E. Mutation analysis

To compare test approaches, we inject faults with mutation operators [25]–[27]. Detection of mutants correlates with detection of real faults [28].

III. How DSP libraries test themselves

To make P1–P3 concrete, we analysed the syntax trees of the test suites of `numpy.fft`, `scipy.fft`, `scipy.signal`, librosa [29] and the functional module of torchaudio (script `benchmarks/survey_test_suites.py`, results in `survey_results.json`). Table I summarises the counts. They are syntactic heuristics: a comparison is counted when a known assertion function is called, and its tolerance when a literal is passed as `rtol`, `atol`, `decimal` or similar.

No test uses Hypothesis or any other input generator. Hand-written tolerances are frequent and heterogeneous, which reflects P3: each test author picks a value that makes the test pass on their machine. Empty inputs appear in a few tests, but the combination that exposed the `lfilter` and `sosfilt` faults appears in 2 tests only.

IV. Contribution

dsp-props is a layer on top of Hypothesis. The user declares *what an operator is*, and the library derives the relations, the generators and the tolerance. For instance, `@dp.lti_filter(transfer=(b, a))` declares a causal linear time-invariant (LTI) filter with transfer function $B(z)/A(z)$; `dp.verify` then runs every applicable relation on generated inputs and returns a report with the smallest failing input found by shrinking.

A. Relations (answer to P1)

Table II lists the relations per operator class. Each one is a consequence of the declaration and is proved in Appendix A; we state them on finite sequences of length N, as implementations compute them, which matters for shifts and edges. The relations replace the oracle: no expected output is needed, except for the optional transfer function, which the user states once instead of typing expected arrays.

B. Generators (answer to P2)

Signals are drawn from seven families: impulses, constants, sinusoids at arbitrary frequency (including off-grid ones), sinusoids near the Nyquist frequency, pairs of tones less than one DFT bin apart, sparse signals and white noise; amplitudes are drawn from $\{10^{-100}, 10^{-3}, 1, 10^{3}, 10^{100}\}$. Lengths are biased towards 1, 2, small primes, and $2^k - 1$, 2^k, $2^k + 1$, and block splits include empty and one-sample blocks. The families are ordered so that shrinking converges to short impulses, which yields readable counterexamples.

C. Tolerances (answer to P3)

All comparisons use the ℓ_2 norm of the error, scaled by the norm of a reference quantity, and a model of the admissible error.

TABLE I
TEST SUITES OF FIVE DSP LIBRARIES (SYNTACTIC COUNTS). TOL.: COMPARISONS WITH A HAND-WRITTEN TOLERANCE; DISTINCT: NUMBER OF DISTINCT TOLERANCE VALUES; EMPTY: TESTS WITH AN EMPTY INPUT; ZI+EMPTY: TESTS THAT ALSO PASS AN INITIAL STATE

Suite	Tests	Hypothesis	Comparisons	Tol.	Distinct	Range	Empty	zi+empty
NumPy (numpy.fft)	52	0	102	66	3	10^{-14}–10^{-6}	1	0
SciPy (scipy.fft)	96	0	132	11	7	10^{-10}–3.0×10^{-6}	0	0
SciPy (scipy.signal)	1095	0	1645	584	66	10^{-31}–3.01	62	2
librosa	916	0	664	60	20	10^{-12}–3	4	0
torchaudio (functional)	229	0	13	12	12	10^{-7}–0.1	0	0

TABLE II
RELATIONS CHECKED FOR EACH OPERATOR CLASS

Declaration	Relations
Transform (DFT, real DFT, generic linear)	linearity; circular shift theorem; Parseval's identity (scale declared); Hermitian symmetry and real DC/Nyquist bins of a real input; inversion; convolution theorem
LTI filter	linearity; time invariance (delay with zero state); causality; frequency response equal to the declared B/A; output bounded by a declared gain for every input bounded by 1
Rational resampler L/M	linearity; output length $\lceil NL/M \rceil$; delay of M input samples gives a delay of L output samples; DC gain; rejection of tones above the output Nyquist frequency
Streaming / stateful	any split of the input into blocks, including empty and one-sample blocks, gives the one-shot output

FFT. For transforms, $\|\hat{y}-y\|_2 \le c\,\varepsilon\,\max(\log_2 N, 1)\,\|y\|_2 + N\,\mathrm{tiny}$, the form of the forward error bound of the radix-2 FFT [24, Thm. 24.2], with a constant c that covers the few chained transforms in a relation, and a floor at the smallest normal number tiny that keeps exact zeros strict while tolerating underflow.

Linear recursions. For a filter $y = (B/A)x$ computed by a direct-form recursion, Appendix B derives

$$\|\hat{y} - y\|_2 \le c\,\varepsilon\,k\,\|h_A\|_1\big(\|b\|_1 + \|a_{1:}\|_1\|h_{B/A}\|_1\big)\|x\|_2, \quad (1)$$

to first order in ε, where k is the number of coefficients, h_A and $h_{B/A}$ are the impulse responses of $1/A$ and B/A, and $a_{1:}$ are the feedback coefficients. The bound grows as poles approach the unit circle, which is where recursions lose accuracy.

Calibration. An analytic bound cannot describe every library: an FFT may switch algorithm with the factorisation of N. `dp.calibrate` measures the error of an implementation against an extended-precision reference for every length and turns it into a per-length tolerance. It refuses to calibrate when the error exceeds what rounding can explain ($10^6\varepsilon$), so a fault cannot be absorbed into the tolerance.

D. Differences with the state of the art

Compared with generic PBT, dsp-props supplies the three pieces of domain knowledge that PBT leaves to the user; Section V-A measures the contribution of two of them. Compared with self-testing of the DFT [16], [20], it covers filters, resamplers and streaming code, and its tolerance follows the operator. Compared with differential testing [21], it needs a single implementation. Compared with the test suites of Section III, it needs no expected values and no hand-chosen tolerance.

V. RESULTS

We answer four research questions (RQ). **RQ1**: does dsp-props detect more faults than the state of practice? **RQ2**: what do the domain generators and the error-model tolerances contribute? **RQ3**: is dsp-props itself correct, i.e., free of false alarms, with sound references and generators? **RQ4**: does it find faults in real libraries?

Environment: Python 3.13.16, NumPy 2.5.4, SciPy 1.18.1, Hypothesis 6.169.3, on a 4-core Linux x86-64 machine. All runs use `max_examples=200` per relation, the library default.

A. RQ1 and RQ2: fault detection on generated mutants

Subjects: We wrote four small reference implementations with the interface of a library function (`benchmarks/comparison/subjects/`): a radix-2 FFT with Bluestein's algorithm for other lengths, a direct-form II transposed `lfilter` with initial state, a polyphase `upfirdn` used by a port of `resample_poly`, and a block FIR filter by overlap-add. All four agree with NumPy or SciPy to a relative error below 1.6×10^{-15} and pass every suite below.

Faults: `mutate.py` generates every first-order mutant with the classic operators [25]: arithmetic, bitwise and relational operator replacement, constant replacement ($k \to k \pm 1$, $c \to c + 1$), unary minus deletion, `min`/`max` swap and statement deletion, giving 183 mutants (76 FFT, 55 `lfilter`, 36 `upfirdn`, 16 overlap-add).

Ground truth: An oracle used only for classification runs each mutant on a fixed battery of inputs (all lengths up to 130 and several larger ones for the FFT, many filters, including order 0, and initial states for `lfilter`, rates up to $5/4$ and $1/7$ for `upfirdn` with and without default arguments, random block splits for overlap-add) and compares with an independent reference. A mutant is *equivalent* if its largest relative error is at most 10^{-12}, an *accuracy* fault if it is between 10^{-12} and 10^{-6}, and a *functional* fault above 10^{-6}, or if it raises, returns the wrong shape or exceeds the time limit. This gives 166 functional faults and 17 equivalent mutants. No mutant falls in the accuracy band: the largest relative error of a

mutant is either below 10^{-14} or above 1. First-order mutation of these subjects thus produces gross faults only, and we study accuracy faults with the hand-written mutants below.

Approaches: SoP (state of practice) is the test suite of the library whose interface the subject implements, ported test by test with its inputs and tolerances: NumPy's and SciPy's FFT tests, SciPy's `lfilter`, `lfilter_zi`, `upfirdn` and `resample_poly` tests, including the regression test added for gh-22571; the overlap-add subject, which has no library counterpart, gets four tests in the same style. *dsp-props* runs the declarations a user would write (two to four per subject). Two ablations isolate the components: *uniform* replaces the domain generators by uniform lengths in $[1, 512]$ and values in $[-1, 1]$, and *fixed-tol* replaces every tolerance model by a fixed relative tolerance of 10^{-7} (the default of `numpy.testing.assert_allclose`); *generic PBT* applies both, i.e., the same relations written with Hypothesis alone. Each randomised approach runs with 5 seeds; a run that exceeds 45 s counts as a detection, since a hanging test fails in continuous integration.

RQ1: Table III gives the share of mutants detected, averaged over seeds. dsp-props detects 90 % of the 166 functional mutants and the ported suites 96 %; no approach raises an alarm on an equivalent mutant. The two sets of detected faults are not nested: 17 mutants are detected only by the suites, 6 only by dsp-props, and together they detect 100 % of them.

Every miss of dsp-props has the same cause: the fault shows only for operator parameters that no declaration uses. A declaration fixes the coefficients of a filter or the rates of a resampler, and dsp-props then varies the input signal. Seven `lfilter` mutants break filters of order 0 or 1, or filters whose numerator is shorter than the denominator, and the declared filters have orders 4 and 2 with numerators at least as long. The `upfirdn` survivors change the length of the full output, which the declarations never see since they reach `upfirdn` through `resample_poly` and through a FIR filter truncated to the input length, or ignore the last input sample, which every relation sees identically on both of its sides. The suites miss the opposite kind of fault. Their only comparisons with expected values on the FFT use length 30, which the subject computes with Bluestein's algorithm; on powers of two they only check that `ifft` inverts `fft`, which two mutants of the radix-2 twiddle factors still satisfy. They also always pass the rates of `upfirdn` explicitly, so four mutants of its default arguments survive.

dsp-props therefore does not detect more of these faults than suites refined over many years; it detects other ones, without expected values, and its misses are a matter of declarations rather than of relations: declaring a first-order filter would have caught the `lfilter` survivors.

Hand-written faults: The repository also contains 31 hand-written faults in six families, closer to real bugs (a forgotten $1/N$, a twiddle sign error, single-precision intermediates, a filter that changes after 64 samples, a gain wrong by 1 %, a missing interpolation gain, state reset between blocks; script `mutants/run_mutants.py`). The four single-precision faults are accuracy faults in the sense above. dsp-props detects all of them; 27 fail within the first three generated inputs, and the median length of the shrunk counterexample is 2 samples. One fault class is a documented blind spot: a filter whose gain is wrong by 1% is still linear, time-invariant and causal, and is detected only when its transfer function is declared.

RQ2: On the generated mutants the four randomised variants detect the same faults, up to one FFT mutant that the variants with uniform inputs miss in some seeds (Table III). These faults are gross: the median number of inputs before the first failure is 3 for dsp-props and 2 for uniform inputs, and the median counterexample has 3 and 2 samples. The generators and tolerances do not change what is detected here, and the generators cost time, with a median of 3.21 s per mutant against 1.6 s for uniform inputs and 0.001 s for a ported suite.

The hand-written faults, which include the accuracy faults, separate the variants (Table IV). Every variant detects every fault in every seed, but not equally fast. With the domain generators, the median failure appears at input 3 (90th percentile 7); with uniform inputs at input 8 (22), and with generic PBT at input 8 (39). Counterexamples are smaller too: the 90th percentile of their length is 17 samples for dsp-props, 34 with uniform inputs, and 97 with a fixed tolerance. Finally, generic PBT raised a false alarm on the correct real FFT, in 1 of its 5 runs: Hypothesis drew a subnormal input of norm 10^{-323}, for which a purely relative tolerance is zero and a rounding error of one subnormal fails the test. The error model of dsp-props includes an absolute floor, and it raised no false alarm. With 200 inputs per relation, these differences do not change the detection rate; with a smaller budget, or on faults that need a specific input, they would.

B. RQ3: is the tool itself correct?

A reviewer of any fault-finding tool should ask whether its alarms are real. We check four things (script `benchmarks/validation.py`, results in `validation.json`).

Relations: Every relation is a theorem about the declared class of operators (Appendix A), so a correct implementation in exact arithmetic never violates it; only the floating-point comparison can raise a false alarm.

False alarms: We ran every declaration on correct implementations, NumPy's FFT and real FFT, SciPy's `lfilter`, `sosfilt`, `upfirdn` and `resample_poly`, and the four reference subjects, with 20 independent seeds: 2 000 runs of a relation, 248 000 floating-point comparisons, no failures. SciPy's `lfilter` with an initial state on empty blocks is excluded from this count because it is the known fault gh-22571; dsp-props flags it in 5 of 5 runs on SciPy 1.18.1.

Margin: During the same runs we recorded the ratio of the observed error to the tolerance for every comparison. 63 % of the comparisons are exact (ratio 0); the 99th percentile is 0.038 and the maximum 0.20: the tolerance is 5 times larger than the worst error observed on correct code. Table IV shows

TABLE III

MUTANTS DETECTED (%, MEAN OVER SEEDS, WITH MIN–MAX WHEN SEEDS DISAGREE). FOR EQUIVALENT MUTANTS THE FIGURE IS A FALSE-ALARM RATE AND SHOULD BE 0. SOP: PORTED TEST SUITES OF NUMPY AND SCIPY; UNIFORM: DSP-PROPS WITHOUT ITS GENERATORS; FIXED-TOL: WITHOUT ITS TOLERANCE MODELS; GENERIC PBT: WITHOUT BOTH

Subject	Class (mutants)	SoP	dsp-props	uniform	fixed-tol	generic PBT
FFT	functional (63)	97	100	100 (98–100)	100	100 (98–100)
FFT	equivalent (13)	0	0	0	0	0
`lfilter`	functional (51)	100	86	86	86	86
`lfilter`	equivalent (4)	0	0	0	0	0
`upfirdn`	functional (36)	89	72	72	72	72
overlap-add	functional (16)	100	100	100	100	100
all	functional (166)	96	90	90 (89–90)	90	90 (89–90)
all	equivalent (17)	0	0	0	0	0

TABLE IV

HAND-WRITTEN MUTANTS BY FAMILY: RUNS THAT REPORT A FAILURE, OVER MUTANTS AND 5 SEEDS. A CORRECT REFERENCE MUST NEVER FAIL

Mutant	dsp-props	uniform	fixed-tol	generic PBT
FFT, reference	0/5	0/5	0/5	0/5
FFT, 8 mutants	40/40	40/40	40/40	40/40
real FFT, reference	0/5	0/5	0/5	1/5
real FFT, 5 mutants	25/25	25/25	25/25	25/25
DCT, reference	0/5	0/5	0/5	0/5
DCT, 3 mutants	15/15	15/15	15/15	15/15
IIR filter, reference	0/5	0/5	0/5	0/5
IIR filter, 5 mutants	25/25	25/25	25/25	25/25
resampler, reference	0/5	0/5	0/5	0/5
resampler, 6 mutants	30/30	30/30	30/30	30/30
streaming, reference	0/5	0/5	0/5	0/5
streaming, 4 mutants	20/20	20/20	20/20	20/20
mutants always detected	31	31	31	31
first failure, median (p90)	3 (7)	8 (22)	3 (10)	8 (39)
counterexample length, p90	17	34	97	35

that it is nevertheless tight enough to catch the single-precision faults.

References: Calibration and the tests of the IIR model compare against extended-precision references, a DFT computed by NumPy in `long double` and a direct recursion in `long double`. Against a 50-digit computation with mpmath, their relative error is at most $1.0 \times 10^{-3}\,\varepsilon$ and $5.4 \times 10^{-3}\,\varepsilon$ respectively, where ε is the double-precision unit roundoff: the references are three orders of magnitude more accurate than what they are used to measure.

Generators: We sampled each generator and checked its claims. All 19 pathological lengths are produced, impulses have exactly one non-zero sample, all signals are finite and of the requested length, and amplitudes cover 10^{-100} to 10^{100}; 78 % of block splits contain an empty block and 37 % a one-sample block; near-Nyquist frequencies lie in $[0.45, 0.5]$; 99 % of the tone pairs are less than one bin apart. The audit also found a weakness: the "off-grid" sinusoid lands exactly on a DFT bin in 39 % of draws, because Hypothesis favours simple values such as 0 and 0.5 (23 % of draws). These are useful edge cases, but the family is less often off-grid than its name says; a future version will draw both on-grid and strictly off-grid frequencies.

C. RQ4: faults in real libraries

A campaign over 46 targets in SciPy, NumPy, PyTorch, librosa and torchaudio (`benchmarks/campaign.py`) produced the following findings.

- `scipy.signal.lfilter` returns an uninitialised final state, and `sosfilt` raises, when given an empty block with an initial state. The `lfilter` part is an independent rediscovery of gh-22571, fixed in the development version after SciPy 1.18.1; the `sosfilt` part, which fails on any input that is empty along the filtered axis, was new. The same fault makes librosa's streaming pre-emphasis fail on an empty block.
- `scipy.signal.tf2sos` drops pure delays: for $y[n] = x[n-2]$ it returns an identity section, so `sosfilt(tf2sos(b, a), x)` differs from `lfilter(b, a, x)`.
- torchaudio's `deemphasis`, documented as the inverse of `preemphasis`, clamps its output to $[-1, 1]$.
- PyTorch's CPU FFT reaches $195\,\varepsilon$ for $N = 596$, against at most $4.1\,\varepsilon$ for NumPy over $N = 2, \ldots, 1200$ (Fig. 1); this is not a fault, but it shows why one fixed tolerance cannot fit every library.

The campaign also produced three alarms that triage traced to errors in our own specifications (e.g., `filtfilt` is not symmetric under time reversal with its default padding); they are listed in the repository.

D. Threats to validity

Mutants are a proxy for real faults [28]; the hand-written faults and the bugs of RQ4 complement them. The SoP suites are ports of real suites restricted to the subject's interface; the original suites also test other dtypes and shapes. The subjects are small, and the oracle thresholds (10^{-12}, 10^{-6}) are a choice; per-mutant errors are in the result file so that other thresholds can be applied. The oracle was extended once after the first run: it did not call `upfirdn` with its default arguments nor `lfilter` with an order-0 filter, so it labelled as equivalent eight mutants that the SoP suites or dsp-props detected; with the extended oracle, no approach detects any equivalent mutant. The kill rates of dsp-props depend on the declarations, which we wrote before running the mutants and

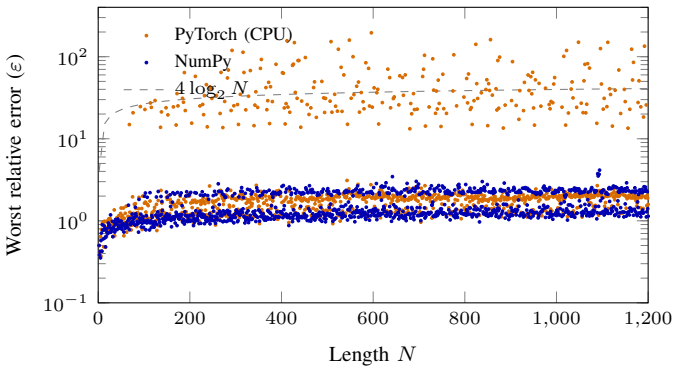


Fig. 1. Worst relative FFT error, in units of ε, versus length N, against a `long double` reference (`benchmarks/error_profile.py`). SciPy, not shown, behaves like NumPy.

did not change afterwards. Results depend on random seeds, which we vary and report.

Reproducing: The scripts `run_comparison.py` and `run_hand_mutants.py` (in `benchmarks/comparison/`), `benchmarks/validation.py`, `mutants/run_mutants.py` and `benchmarks/survey_test_suites.py` produce the result files, and `paper/article/make_numbers.py` turns them into every number and table of this article.

VI. Conclusion

DSP code is tested against expected values that someone computed elsewhere, with tolerances chosen case by case, on inputs that rarely include the cases that break it. Generic property-based and metamorphic testing remove the need for expected values but leave the relations, the inputs and the tolerances to each developer. dsp-props supplies these three pieces for transforms, filters, resamplers and streaming code. On 166 generated faults it detects 90 %, against 96 % for the ported test suites of NumPy and SciPy, and the two are complementary: together they detect 100 %. Its misses come from operator parameters that no declaration exercised; the misses of the suites, from code paths that their few expected values do not reach. It raised no false alarm on correct implementations, its references and generators were audited, and the audit itself revealed a weakness that we report.

We reported the faults found to the projects concerned: the empty-block fault of `lfilter` and `sosfilt` to SciPy (issue #26406, the `lfilter` part being a duplicate of gh-22571), with a proposed fix (pull request #26415), the delay dropped by `tf2sos` to SciPy, and the clamping of `deemphasis` to torchaudio.

Acknowledgments

The authors thank Thomas Calmant for feedback on the structure of this article and on its experimental protocol.

AI usage disclosure

The authors conceived and directed this research: they defined the problem, the scope and the design of dsp-props, chose the experimental protocol, checked every result and wrote the bug reports to upstream projects. An AI coding assistant (Claude, Anthropic) was used under their direction to write code and experiment scripts and to assist with the writing. Every number in this article is produced by a script of the repository.

Conflict of interest

The authors declare no conflict of interest.

References

[1] C. R. Harris, K. J. Millman, S. J. van der Walt *et al.*, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020.

[2] P. Virtanen, R. Gommers, T. E. Oliphant *et al.*, "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, no. 3, pp. 261–272, 2020.

[3] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, vol. 32, 2019. [Online]. Available: https://papers.nips.cc/paper_files/paper/2019/file/bdbca288fee7f92f2bfa9f7012727740-Paper.pdf

[4] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, "The oracle problem in software testing: A survey," *IEEE Transactions on Software Engineering*, vol. 41, no. 5, pp. 507–525, 2015.

[5] U. Kanewala and J. M. Bieman, "Testing scientific software: A systematic literature review," *Information and Software Technology*, vol. 56, no. 10, pp. 1219–1232, 2014.

[6] K. Claessen and J. Hughes, "QuickCheck: A lightweight tool for random testing of Haskell programs," in *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming (ICFP '00)*. ACM, 2000, pp. 268–279.

[7] D. R. MacIver, Z. Hatfield-Dodds, and Many Contributors, "Hypothesis: A new approach to property-based testing," *Journal of Open Source Software*, vol. 4, no. 43, p. 1891, 2019.

[8] T. Y. Chen, F.-C. Kuo, H. Liu, P.-L. Poon, D. Towey, T. H. Tse, and Z. Q. Zhou, "Metamorphic testing: A review of challenges and opportunities," *ACM Computing Surveys*, vol. 51, no. 1, pp. 4:1–4:27, 2018.

[9] S. Segura, G. Fraser, A. B. Sánchez, and A. Ruiz-Cortés, "A survey on metamorphic testing," *IEEE Transactions on Software Engineering*, vol. 42, no. 9, pp. 805–824, 2016.

[10] T. Y. Chen, J. W. K. Ho, H. Liu, and X. Xie, "An innovative approach for testing bioinformatics programs using metamorphic testing," *BMC Bioinformatics*, vol. 10, p. 24, 2009.

[11] F. Jafari and A. Nadeem, "Measuring effectiveness of metamorphic relations for image processing using mutation testing," *Journal of Imaging*, vol. 10, no. 4, p. 87, 2024.

[12] D. R. MacIver and A. F. Donaldson, "Test-case reduction via test-case generation: Insights from the Hypothesis reducer," in *34th European Conference on Object-Oriented Programming (ECOOP 2020)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), R. Hirschfeld and T. Pape, Eds., vol. 166. Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2020, pp. 13:1–13:27.

[13] H. Goldstein, J. W. Cutler, D. Dickstein, B. C. Pierce, and A. Head, "Property-based testing in practice," in *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering (ICSE '24)*. ACM, 2024.

[14] M. Maaz, L. DeVoe, Z. Hatfield-Dodds, and N. Carlini, "Agentic property-based testing: Finding bugs across the Python ecosystem," 2025, also presented at the NeurIPS 2025 Workshop on Deep Learning for Code in the Agentic Era.

[15] M. Blum, M. Luby, and R. Rubinfeld, "Self-testing/correcting with applications to numerical problems," *Journal of Computer and System Sciences*, vol. 47, no. 3, pp. 549–595, 1993.

[16] F. Ergün, "Testing multivariate linear functions: Overcoming the generator bottleneck," in *Proceedings of the Twenty-Seventh Annual ACM Symposium on Theory of Computing (STOC '95)*. ACM, 1995, pp. 407–416.
[17] F. Ergün, S. R. Kumar, and D. Sivakumar, "Self-testing without the generator bottleneck," *SIAM Journal on Computing*, vol. 29, no. 5, pp. 1630–1651, 2000.
[18] F. Ergün, S. R. Kumar, and R. Rubinfeld, "Checking approximate computations of polynomials and functional equations," *SIAM Journal on Computing*, vol. 31, no. 2, pp. 550–576, 2001.
[19] M. Frigo and S. G. Johnson, "The design and implementation of FFTW3," *Proceedings of the IEEE*, vol. 93, no. 2, pp. 216–231, 2005.
[20] ——, "FFTW 3 source code, file `libbench2/verify-lib.c` (self-test used by `bench -verify`)," https://github.com/FFTW/fftw3/blob/master/libbench2/verify-lib.c, accessed 15 October 2026.
[21] J. Vanover, X. Deng, and C. Rubio-González, "Discovering discrepancies in numerical libraries," in *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '20)*. ACM, 2020, pp. 488–501.
[22] A. Di Franco, H. Guo, and C. Rubio-González, "A comprehensive study of real-world numerical bug characteristics," in *Proceedings of the 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE 2017)*. IEEE, 2017, pp. 509–519.
[23] J. C. Schatzman, "Accuracy of the discrete Fourier transform and the fast Fourier transform," *SIAM Journal on Scientific Computing*, vol. 17, no. 5, pp. 1150–1166, 1996.
[24] N. J. Higham, *Accuracy and Stability of Numerical Algorithms*, 2nd ed. Philadelphia, PA: Society for Industrial and Applied Mathematics (SIAM), 2002.
[25] A. J. Offutt, A. Lee, G. Rothermel, R. H. Untch, and C. Zapf, "An experimental determination of sufficient mutant operators," *ACM Transactions on Software Engineering and Methodology*, vol. 5, no. 2, pp. 99–118, 1996.
[26] Y. Jia and M. Harman, "An analysis and survey of the development of mutation testing," *IEEE Transactions on Software Engineering*, vol. 37, no. 5, pp. 649–678, 2011.
[27] M. Papadakis, M. Kintis, J. Zhang, Y. Jia, Y. Le Traon, and M. Harman, "Mutation testing advances: An analysis and survey," in *Advances in Computers*. Elsevier, 2019, vol. 112, pp. 275–378.
[28] R. Just, D. Jalali, L. Inozemtseva, M. D. Ernst, R. Holmes, and G. Fraser, "Are mutants a valid substitute for real faults in software testing?" in *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE 2014)*. ACM, 2014, pp. 654–665.
[29] B. McFee, C. Raffel, D. Liang, D. P. W. Ellis, M. McVicar, E. Battenberg, and O. Nieto, "librosa: Audio and music signal analysis in Python," in *Proceedings of the 14th Python in Science Conference (SciPy 2015)*, 2015, pp. 18–24.

APPENDIX A
PROOFS OF THE RELATIONS

Sequences have length N, indices are taken modulo N for the DFT, and $\mathcal{F}x[k] = \sum_{n=0}^{N-1} x[n]\,\omega^{kn}$ with $\omega = e^{-2\pi i/N}$.

Proposition 1 (Linearity)**.** *$\mathcal{F}$ is linear, and so is every LTI filter from zero state and every resampler of the form "upsample by L, filter, downsample by M".*

Proof. Each is a composition of linear maps: a matrix product for $\mathcal{F}$; zero insertion, convolution with h and decimation for the resampler; for a filter from zero state, y is the convolution $h * x$ truncated to N samples. □

Proposition 2 (Circular shift)**.** *If $x_s[n] = x[n-s]$, then $\mathcal{F}x_s[k] = \omega^{ks}\mathcal{F}x[k]$.*

Proof. $\sum_n x[n-s]\omega^{kn} = \sum_m x[m]\omega^{k(m+s)}$ with $m = n-s$ running over all residues. □

Proposition 3 (Parseval)**.** $\|\mathcal{F}x\|_2^2 = N\|x\|_2^2$.

Proof. $\mathcal{F} = \sqrt{N}\,U$ with U unitary, since $\sum_k \omega^{k(n-m)} = N\delta_{nm}$. □

A transform declared with another normalisation states its scale, which is checked instead of N.

Proposition 4 (Hermitian symmetry)**.** *If x is real, $\mathcal{F}x[N-k] = \overline{\mathcal{F}x[k]}$; in particular $\mathcal{F}x[0]$ and, for even N, $\mathcal{F}x[N/2]$ are real.*

Proof. $\omega^{(N-k)n} = \overline{\omega^{kn}}$ and x is real. For $k = 0$ and $k = N/2$, the bin is its own conjugate. □

Proposition 5 (Inversion and convolution)**.** *$\mathcal{F}^{-1}\mathcal{F}x = x$, and $\mathcal{F}^{-1}(\mathcal{F}x \cdot \mathcal{F}y) = x \circledast y$, the circular convolution.*

Proof. The first follows from $\mathcal{F}^{-1} = \frac{1}{N}\overline{\mathcal{F}}$ and Parseval's orthogonality. For the second, $\mathcal{F}(x \circledast y)[k] = \sum_n \sum_m x[m]y[n-m]\omega^{kn} = \mathcal{F}x[k]\,\mathcal{F}y[k]$ by the shift proposition. □

dsp-props computes $x \circledast y$ directly in $O(N^2)$, independently of the operator under test.

Proposition 6 (Time invariance and causality on finite sequences)**.** *Let f be a causal LTI filter applied from zero state, $f(x)[n] = \sum_{m \le n} h[n-m]x[m]$. Then (i) $f([0_s, x]) = [0_s, f(x)]$ on $N+s$ samples, and (ii) if x and x' agree on $[0,k)$, then $f(x)$ and $f(x')$ agree on $[0,k)$.*

Proof. (i) For $n < s$ the sum only involves zeros; for $n \ge s$, substitute $m' = m - s$. (ii) $f(x)[n]$ for $n < k$ only involves $x[m]$, $m \le n < k$. □

For a non-causal filter (e.g., a centred FIR), (i) holds after discarding the first s output samples, which is what the library checks.

Proposition 7 (Transfer function)**.** *For a stable causal filter with transfer function $H = B/A$ and impulse response h, the length-L DFT of $(h[0], \dots, h[L-1])$ equals $H(e^{2\pi ik/L})$ up to the aliasing term $\sum_{n \ge L} h[n]\omega_L^{kn}$, whose norm is bounded by $\sum_{n \ge L} |h[n]|$.*

Proof. $H(e^{i\theta}) = \sum_{n \ge 0} h[n]e^{-i\theta n}$ converges absolutely for a stable filter; split the sum at L. □

The library chooses L so that $r^L < 10^{-17}$, with r the largest pole modulus, and adds the tail to the tolerance.

Proposition 8 (Bounded gain)**.** *For an LTI filter with $\|h\|_1 \le G$, every input with $|x[n]| \le 1$ gives $|y[n]| \le G$, and the input $x[m] = \operatorname{sign}(h[N-1-m])$ attains $\sum_{n<N} |h[n]|$ at $n = N-1$.*

Proof. $|y[n]| \le \sum_m |h[n-m]||x[m]| \le \|h\|_1$; with the given input, every term of $y[N-1] = \sum_m h[N-1-m]x[m]$ is $|h[N-1-m]|$. □

Proposition 9 (Periodic time invariance)**.** *Let $f = \downarrow_M \circ (h * \cdot) \circ \uparrow_L$. Delaying the input by Ms samples delays the output by Ls samples.*

Proof. $\uparrow_L$ turns a delay of Ms into a delay of LMs; the filter preserves it; $\downarrow_M$ turns a delay of LMs, a multiple of M, into a delay of Ls. □

The output length $\lceil NL/M \rceil$, the DC gain and the alias rejection are declared specifications rather than theorems; their tolerances (ripple, stop-band level) are part of the declaration.

Proposition 10 (Block invariance)**.** *Let a streaming processor be a state machine $(q_{t+1}, y_t) = \phi(q_t, x_t)$ applied sample by sample, and let* `process` *on a block apply ϕ to its samples in order, starting from the current state and storing the final one. Then for any split $x = x^{(1)} \cdots x^{(J)}$, including empty blocks,* concat_j `process`$(x^{(j)})$ $=$ `process`(x) *from the same initial state.*

Proof. By induction on J: an empty block leaves the state unchanged and outputs nothing, and processing $x^{(1)}$ then $x^{(2)}$ applies ϕ to the samples of $x^{(1)}x^{(2)}$ in order. □

Block-based algorithms such as overlap-add are not sample-by-sample machines, but a correct one computes the same function, which is what the relation checks.

APPENDIX B
ERROR BOUND OF A LINEAR RECURSION

Normalise $a_0 = 1$. The direct recursion computes $y[n] = \sum_{j=0}^{q} b_j x[n-j] - \sum_{j=1}^{p} a_j y[n-j]$ with $k = p + q + 1$ products. In floating point, with the standard model [24], the computed $\hat{y}$ satisfies $\hat{y}[n] = \sum_j b_j x[n-j] - \sum_j a_j \hat{y}[n-j] + \delta[n]$ with $|\delta[n]| \le \gamma_k\big(\sum_j |b_j||x[n-j]| + \sum_j |a_j||\hat{y}[n-j]|\big)$, $\gamma_k = k\varepsilon/(1-k\varepsilon)$. The error $e = \hat{y} - y$ therefore obeys $A(z)e = \delta$, i.e., $e = h_A * \delta$. By Young's inequality $\|e\|_2 \le \|h_A\|_1 \|\delta\|_2$, and $\|\delta\|_2 \le \gamma_k(\|b\|_1\|x\|_2 + \|a_{1:}\|_1\|\hat{y}\|_2)$. Finally $\|\hat{y}\|_2 \le \|h_{B/A}\|_1\|x\|_2 + \|e\|_2$, which gives (1) to first order in ε, with $\gamma_k \approx k\varepsilon$. The library uses $c = 4$ to cover the two computations compared in a relation, and the validation of Section V-B confirms that the bound is never reached on correct code.